

Implementação de um Algoritmo de Colônia de Formigas em Ambiente Distribuído

Caroline Freitas Alvernaz¹, Gabriela Lacerda Muniz¹, Giovanna Naves Ribeiro¹,
Guilherme Oliveira de Rodrigues¹, Ji Xinyi¹, João de Jesus e Avila¹,
João Vitor Mendes Moreira¹, Júlia Rodrigues Vasconcellos Melo¹,
Priscila Andrade de Moraes¹

¹Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Resumo. *Este trabalho apresenta um sistema distribuído baseado no Algoritmo de Colônia de Formigas (ACO) aplicado ao Problema do Caixeiro Viajante (TSP), no qual nós executam buscas locais e compartilham matrizes de feromônio para acelerar a convergência. A comunicação ocorre por mensagens em rede, a coordenação por eleição de líder (algoritmo Bully) e a ordenação dos eventos por relógios de Lamport. O sistema conta com um mecanismo de recuperação de estado após falha do líder, que preserva o conhecimento acumulado pela colônia e é comparado a uma versão centralizada equivalente em tempo de execução, qualidade das soluções e resiliência a falhas.*

1. Explicação do problema escolhido

O problema escolhido é a aplicação do Algoritmo de Colônia de Formigas (ACO) ao Problema do Caixeiro Viajante (TSP) em ambiente distribuído. O TSP consiste em encontrar a menor rota que percorra um conjunto de cidades exatamente uma vez e retorne à origem; o ACO é uma meta-heurística inspirada no comportamento de formigas, que constroem soluções de forma probabilística e reforçam os melhores caminhos por meio de trilhas de feromônio. Na versão distribuída implementada, cada nó simula uma colônia independente, explorando regiões distintas do espaço em paralelo. Periodicamente, os nós compartilham suas matrizes de feromônio, acelerando a convergência para boas soluções.

2. Detalhamento da arquitetura do sistema

O sistema é composto por cinco nós distribuídos, executados na mesma máquina como processos independentes, identificados por um ID (1 a 5) e associados a uma porta TCP fixa ($5000+i$). A comunicação ocorre por sockets TCP em topologia totalmente conectada (*full mesh*), permitindo que qualquer nó envie mensagens diretamente a qualquer outro.

2.1. Papéis dos nós

Os papéis são definidos pelo algoritmo de eleição de líder. O Nó 5 (maior ID) assume a liderança inicial, coordenando a sincronização global da matriz de feromônio e, ao assumir o cargo por eleição, a recuperação de estado da colônia. Os demais nós atuam como *workers*: executam o ACO localmente, respondem às solicitações do líder e monitoram sua disponibilidade. Em caso de falha, assume a liderança o nó vivo de maior ID.

2.2. Formato das mensagens

Todas as mensagens seguem uma estrutura JSON com quatro campos: **tipo** (finalidade da mensagem, ex. ELEICAO), **remetente_id** (nó emissor), **timestamp_lamport** (valor do relógio lógico) e **conteudo**. Os tipos RECUPERACAO e RECUPERACAO_RESPOSTA fazem parte do mecanismo de recuperação de estado pós-falha do líder.

2.3. Comunicação entre os nós

A comunicação é organizada em quatro fluxos principais:

- **Eleição de líder:** ao detectar a falha do líder, um nó envia `ELEICAO` aos nós de ID maior, estes respondem `OK` e iniciam nova eleição. Sem resposta, o nó se declara líder e envia `LIDER` em broadcast.
- **Sincronização de feromônio:** o líder solicita as matrizes locais dos workers, calcula a média e redistribui o resultado via broadcast.
- **Deteção de falhas:** workers enviam `HEARTBEAT` periódicos ao líder, que responde. A ausência de resposta indica falha e aciona uma nova eleição.
- **Recuperação de estado pós-falha:** ao assumir a liderança pela primeira vez após uma eleição, o novo líder solicita o estado de cada worker conhecido. As respostas, recebidas dentro de uma janela de tempo limitada, são consolidadas e o estado global reconstruído é redistribuído.

2.4. Papel dos módulos

- **rede.py:** comunicação via TCP (envio de mensagens e serialização JSON).
- **aco.py:** lógica do algoritmo ACO, independente da camada distribuída, operando sobre a matriz de feromônio.
- **aco_centralizado.py:** versão sequencial de referência do mesmo ACO, sem dependência de rede ou coordenação, usada nos experimentos comparativos.
- **coordenacao.py:** relógio de Lamport e algoritmo de eleição de líder (Bully).
- **no.py:** integra os módulos, coordenando o loop principal, as threads auxiliares e o roteamento das mensagens.

2.5. Instância do problema

O TSP utilizado considera um conjunto fixo de 16 cidades, com coordenadas bidimensionais definidas no módulo `instancia.py`. A distância entre duas cidades é dada pela métrica euclidiana (Equação 1), arredondada ao inteiro mais próximo, gerando a matriz de distâncias simétrica usada por todos os nós.

$$d(i, j) = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2} \quad (1)$$

3. Detalhamento da implementação dos requisitos escolhidos

3.1. Comunicação, ACO local, coordenação e integração

A comunicação usa sockets TCP: cada nó executa um servidor em thread separada, com fila thread-safe para mensagens recebidas. O envio é feito por conexões independentes com tratamento de falhas, e a leitura é não bloqueante. Na sincronização de feromônio, o líder aguarda respostas dos workers por até 3 segundos.

Cada nó executa uma instância independente do ACO, construindo rotas a partir das matrizes de feromônio e de distâncias, seguida de evaporação e depósito de feromônio nos caminhos de melhor qualidade; essa matriz é o único dado trocado entre os nós.

A coordenação implementa o relógio de Lamport, que garante a ordenação correta dos eventos, e a eleição de líder Bully: mensagens `ELEICAO` aguardam resposta por

cerca de 2 segundos; sem resposta, o nó se declara líder e notifica os demais via `LIDER`. Esses componentes são integrados no loop principal, que processa mensagens e atualiza o estado de coordenação. Threads auxiliares cuidam de tarefas periódicas: heartbeats a cada 5 segundos (falha do líder detectada após três tentativas sem resposta, $\approx 15s$) e sincronização global de feromônio a cada 10 iterações do ACO.

3.2. Recuperação de estado pós-falha

O tratamento de falhas do líder combina detecção via *heartbeat*, eleição pelo algoritmo Bully e um protocolo de recuperação de estado distribuído. Sem esse protocolo, o nó eleito assumiria a coordenação apenas com sua própria matriz local, descartando o conhecimento acumulado pelos demais (equivalente a reiniciar parcialmente a busca a cada troca de liderança). O protocolo é acionado automaticamente sempre que um nó assume a liderança pela primeira vez após uma eleição:

1. o novo líder envia `RECUPERACAO` a cada worker registrado, removendo do grupo ativo quem não puder ser contatado;
2. cada worker responde com `RECUPERACAO_RESPOSTA`, contendo sua matriz de feromônio, melhor rota e melhor distância encontradas até então;
3. o líder aguarda as respostas por uma janela máxima de 3 segundos, removendo do grupo quem não responder;
4. as matrizes recebidas são consolidadas com a matriz local do líder pela **média elemento a elemento** (mesma rotina da sincronização periódica);
5. a matriz resultante substitui a do líder, que também atualiza seu critério de parada com base na melhor rota global conhecida;
6. o líder redistribui a matriz consolidada via `FEROMONIO` em broadcast, e o ciclo normal do ACO é retomado.

Assim, o conhecimento construído coletivamente pela colônia antes da falha não é descartado: a busca prossegue a partir do estado médio dos nós sobreviventes, em vez de recomeçar do zero.

3.3. Versão centralizada de referência

Para comparar o desempenho da versão distribuída, foi implementada uma versão centralizada e sequencial do mesmo ACO, reaproveitando apenas a instância do problema e a lógica de construção de rotas/atualização de feromônio já existentes, executada em processo único, sem comunicação de rede, eleição de líder ou sincronização lógica. O baseline registra tempo total e melhor distância encontrada, permitindo comparação direta com a versão distribuída sob os mesmos parâmetros.

4. Desafios e problemas encontrados durante a implementação

Durante a implementação, alguns desafios exigiram atenção. Na eleição de líder, nós podiam permanecer indefinidamente com o estado de líder porque o sistema já estava marcado como em processo de eleição. A solução foi reiniciar esse estado a cada nova falha. Quando não havia nós de ID maiores disponíveis durante uma eleição, o próprio nó passou a se declarar líder. Na recuperação de estado, foi necessário garantir que o protocolo fosse disparado apenas na transição de worker para líder, e não a cada novo `LIDER` recebido, além de remover do grupo ativo os workers sem resposta dentro de 3s.

5. Papel e utilidade dos relógios lógicos de Lamport

O relógio de cada nó é um contador local, incrementado a cada envio de mensagem e atualizado a cada recebimento segundo a regra $tempo = \max(tempo_local, tempo_recebido) + 1$. Essa regra garante que, se um evento A causou um evento B , o timestamp de A é sempre menor que o de B , estabelecendo uma ordenação causal entre todos os nós. No sistema, esses relógios são empregados em três contextos:

1. **Ordenação das mensagens de eleição:** durante o Bully, os timestamps revelam a causalidade entre ELEICAO, OK e LIDER, evitando que uma mensagem atrasada de uma eleição anterior interfira em uma mais recente.
2. **Sincronização das matrizes de feromônio:** o timestamp de cada matriz difundida (sincronização periódica ou recuperação) permite descartar atualizações desatualizadas recebidas fora de ordem.
3. **Diagnóstico e rastreabilidade de falhas:** os relógios fornecem uma sequência global consistente de eventos, permitindo reconstruir a ordem exata em que heartbeats, eleições e recuperações ocorreram.

Timestamps físicos foram descartados por serem inadequados em múltiplas máquinas: relógios independentes sofrem *drift* (dessincronização gradual), o que poderia ordenar incorretamente mensagens causalmente relacionadas.

6. Experimentos e análise de desempenho

Esta seção apresenta a análise de desempenho do sistema, estruturada em três dimensões: tempo, qualidade das soluções e resiliência.

6.1. Metodologia experimental

Foram executadas 20 repetições de cada versão (centralizada e distribuída), sob os mesmos parâmetros, 16 cidades, 5 formigas, 100 iterações, registrando tempo total e melhor distância encontrada. Adicionalmente, foram realizados 15 testes de tolerância a falhas (5 repetições em cada um de três cenários: falha precoce, intermediária e tardia), encerrando artificialmente o líder para medir tempo de recuperação, correção da eleição e continuidade da busca. Os dados foram organizados em CSV para gerar os gráficos a seguir.

6.2. Desempenho de tempo

A Figura 1 mostra a distribuição do tempo total de execução (100 iterações) nas 20 execuções de cada versão. A versão centralizada completou em $0,0364s \pm 0,0020s$ (IC 95%), e a distribuída em $2,3220s \pm 0,0124s$. O *speedup* médio $S = T_{\text{centralizado}}/T_{\text{distribuído}} \approx 0,016$, ou seja, a distribuída foi cerca de 64 vezes **mais lenta** nessa instância.

Esse resultado é coerente com a lei de Amdahl: a instância (16 cidades, 5 formigas, 100 iterações) é resolvida pela versão sequencial em milissegundos, onde o overhead de comunicação TCP (conexões, serialização JSON e espera de 3s por sincronização) domina o tempo total, já que a fração paralelizável é pequena demais para compensar esse custo fixo. A arquitetura distribuída tende a se tornar vantajosa apenas em instâncias maiores, onde o tempo de computação por nó passa a dominar o de comunicação.

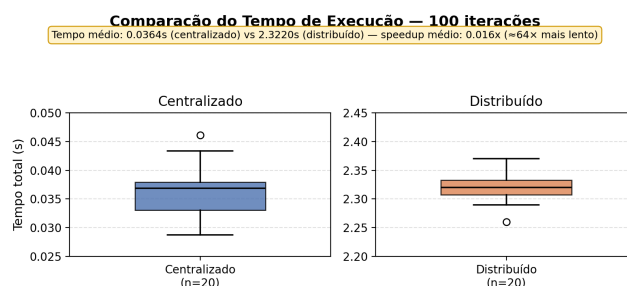


Figura 1. Comparação do tempo total de execução entre as versões centralizada e distribuída. Os painéis usam escalas verticais distintas.

6.3. Qualidade das soluções

A Figura 2 compara a melhor distância encontrada por cada versão ao final das 100 iterações: a centralizada obteve distância média de 73,75, e a distribuída, 73,15, com menor variabilidade entre execuções. A exploração paralela (cada nó com sua colônia, compartilhando conhecimento periodicamente) produziu soluções levemente melhores que a versão sequencial, mesmo com o mesmo total de iterações: a diversidade entre colônias independentes compensa parcialmente o menor número de iterações efetivas por nó.

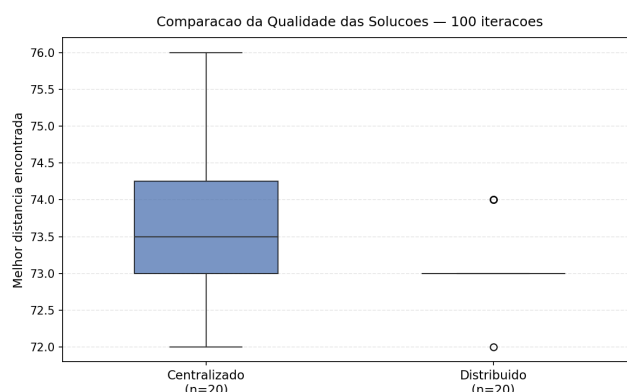


Figura 2. Melhor distância encontrada por versão, 20 execuções cada.

6.4. Resiliência e custo da tolerância a falhas

A versão centralizada não oferece tolerância a falhas: por executar em um único processo, qualquer falha encerra integralmente a execução, sem possibilidade de recuperação. A distribuída paga um custo operacional em troca dessa resiliência, quantificado pelos testes de tolerância a falhas (Figura 3).

Nos 15 testes realizados, a eleição do novo líder ocorreu corretamente em todos os casos e a busca continuou sem reinicialização. O tempo médio de recuperação completa (detecção, eleição Bully e recuperação de estado) foi de 13,49s, sem diferença relevante entre os três momentos de falha testados. Esse tempo é dominado pelo timeout de detecção de 8s, não pelo protocolo de recuperação em si (até 3s). Esse é o trade-off central entre overhead operacional e continuidade do serviço: a distribuída sacrifica desempenho bruto e introduz indisponibilidade temporária na recuperação, mas, diferentemente da centralizada, sobrevive à perda do coordenador sem perder o trabalho acumulado.

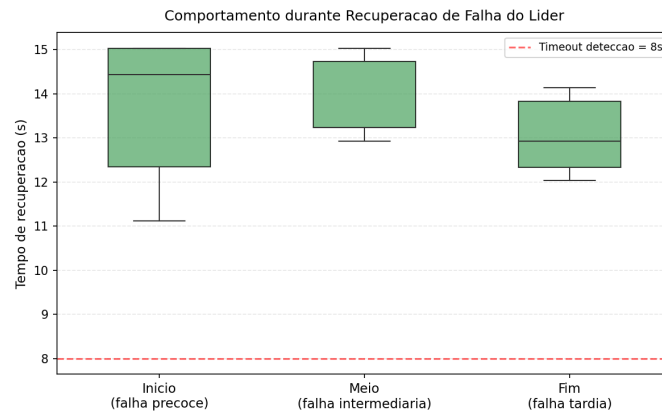


Figura 3. Tempo de recuperação por momento da falha, 5 execuções por cenário.

7. Conclusão

O sistema desenvolvido combina tolerância a falhas, detecção via *heartbeat*, eleição Bully e recuperação de estado que preserva o conhecimento acumulado pela colônia, com uma análise de desempenho que revela que: para instâncias pequenas do problema, o overhead de comunicação em rede supera amplamente o ganho potencial de paralelismo, fazendo da versão distribuída uma opção mais lenta, porém mais resiliente que a sequencial.

Como limitações, destacam-se: ausência de latência de rede real (nós na mesma máquina); instância do TSP pequena, favorecendo a centralizada em tempo; e timeout de detecção (8s) conservador, sem análise sistemática do trade-off com falsos positivos. Como melhorias futuras: testar instâncias maiores para identificar o ponto de inflexão; executar nós em máquinas distintas; e investigar detecção de falha adaptativa.

8. Papel de cada aluno na condução do trabalho

- Caroline Alvernaz: integração dos módulos (sincronização de feromônio e detecção de falha); testes comparativos de tempo, qualidade e tolerância a falhas;
- Gabriela Lacerda: integração dos módulos; redação das seções sobre os relógios de Lamport e análise comparativa entre as arquiteturas; incorporação dos gráficos.
- Giovanna Ribeiro: implementação da camada de comunicação de rede e condução dos experimentos comparativos de desempenho.
- Guilherme Oliveira: implementação da camada de rede, do protocolo de recuperação pós-falha e da consolidação das matrizes dos nós sobreviventes.
- Ji Xinyi: implementação do núcleo do ACO; desenvolvimento do protocolo de recuperação de estado pós-falha do líder.
- João de Jesus: implementação do núcleo do ACO; estabilização da aplicação, tratando falhas de conexão e documentando as instruções de execução.
- João Vitor Mendes: implementação do relógio de Lamport, do algoritmo de eleição Bully e estabilização da aplicação.
- Júlia Rodrigues: integração dos módulos do sistema; construção da versão centralizada e coleta de métricas de desempenho (tempo, speedup e eficiência).
- Priscila Andrade: implementação do relógio de Lamport e do algoritmo de eleição Bully; construção da versão centralizada e coleta de métricas de desempenho.